Bem Lembrado!

UNILASALLE · Quarteto Nem Tão Fantástico

Identidade Visual



1. Proposta

Bem Lembrado! é um sistema em Python orientado a objetos que ajuda cuidadores a registrar quem cuida de quem, quais medicamentos cada paciente usa e se cada dose foi administrada.

Problema. Cuidadores acompanham pacientes e horários de medicamentos. Quando essas informações ficam espalhadas em cadernos e planilhas, é difícil manter um histórico organizado.

Objetivo. Centralizar os registros de pacientes, cuidadores, medicamentos e administrações em um só lugar.

Público-alvo. Cuidadores domiciliares, familiares responsáveis e pequenas equipes de cuidado.

Equipe e responsabilidades.

Integrante	Responsabilidade principal
Gabriel Eninger	Documentação e diagramas
Israel Ribeiro	Definição e criação das classes
Joaquim Teixeira	Definição e criação das classes
Mariana Rocha	Organização, identidade visual, proposta, entradas e saídas

Ferramentas. Visual Studio Code, Python, Git e GitHub, GitHub Projects e Trello (Kanban) e Figma (design).

O que o sistema faz hoje.

- Cadastra um cuidador (nome, idade, telefone e especialidade) e um paciente (nome, idade, telefone e contato de emergência) e apresenta os dados de cada um.
- Cadastra um medicamento com dose, horários, estoque e período de uso (data de início e, opcionalmente, data de fim).

- Informa o estoque atual e se ele está baixo (5 unidades ou menos).
- Registra a administração de um medicamento por um cuidador, com data e hora automáticas, observação opcional e status validado (Realizada, Não realizada ou Pendente).

Entradas e saídas.

- **Entradas:** dados do cuidador, do paciente e do medicamento, e o status e a observação da administração, definidos no `main.py`.
- **Saídas:** texto no terminal com os dados de cada objeto, o estoque atual, a indicação de estoque baixo e o registro de administração com seu status.

2. Diagrama de classes e requisitos de POO

O diagrama de classes representa o código atual: **5 classes** em 5 arquivos `.py`, mais o `main.py` que cria e usa os objetos. (Colar aqui o diagrama gerado no PlantUML a partir do código.)

Classe	Arquivo	Atributos	Métodos
Pessoa	pessoa.py	nome, idade, telefone	apresentar()
Paciente (herda de Pessoa)	paciente.py	contato_de_emergencia	apresentar()
Cuidador (herda de Pessoa)	cuidador.py	especialidade	apresentar()
Medicamentos	medicamento.py	nome, dose, horários, estoque, data_inicio, data_fim (opcional)	consultar_estoque(), verificar_estoque_baixo(), apresentar()
RegistroAdministracao	registros.py	medicamento, cuidador, data_hora, __status (privado), observacao	consultar_status(), apresentar()

Relacionamentos e regras.

- Paciente e Cuidador herdam de Pessoa.
- RegistroAdministracao recebe um Medicamentos e um Cuidador no construtor (associação). A data e a hora são gravadas no momento da criação.
- O status só aceita "Realizada", "Não realizada" ou "Pendente"; qualquer outro valor gera `ValueError`.
- O estoque é considerado baixo quando é menor ou igual a 5.

Requisitos do projeto atendidos.

Requisito	Onde aparece
Pelo menos 5 classes	5 classes (Pessoa, Paciente, Cuidador, Medicamentos, RegistroAdministracao)
Herança	Paciente e Cuidador herdam de Pessoa

Requisito	Onde aparece
Polimorfismo	apresentar() sobrescrito em Paciente e Cuidador, chamando super()
Associação	RegistroAdministracao com Medicamentos e Cuidador
Programa principal com 2+ objetos	main.py cria Cuidador, Paciente, Medicamentos e RegistroAdministracao

3. Telas e interação com o usuário

No estado atual, a interação é por terminal: o `main.py` cria os objetos e imprime cada bloco no console. Ainda não há menu nem interface gráfica.

Tela 1 — Execução do `main.py` no terminal. Mostra, em ordem:

1. **Cuidador:** Israel, 49 anos, telefone (51) 98186-4241, especialidade Enfermagem.
2. **Paciente:** João, 39 anos, contato de emergência Maria.
3. **Medicamento:** Dipirona, 500 mg, horários 08:00, 12:00 e 18:00, estoque 10.
4. **Estoque:** atual 10; "Estoque baixo?" retorna False.
5. **Registro de administração:** Dipirona, cuidador Israel, 29/10/2026 20:36, status Realizada, observação "Administração registrada pelo cuidador".

(Colar aqui o print do terminal.)

Entradas, saídas e persistência (Etapa 2).

Pergunta	Situação no código
Lê algum arquivo?	Não.
Há objetos persistentes?	Não: os objetos existem só em memória durante a execução.
Há menu ou interface gráfica?	Não. A interação é a saída impressa no terminal.
O que é lido do teclado?	Nada: os dados são definidos no `main.py`.
O que o sistema gera de saída?	Texto no terminal: dados do cuidador, do paciente e do medicamento, estoque, indicação de estoque baixo, registro de administração e seu status.

4. Repositório, bibliotecas, CRUD e testes

Repositório remoto. https://github.com/israeldev-stack/projeto-poo

Bibliotecas e classes externas. Somente a biblioteca padrão do Python: o módulo `datetime` (em registros.py, para gravar a data e a hora do registro).

Exemplos de testes. Teste manual executando `main.py`:

Cenário	Entrada	Resultado esperado	Resultado obtido
Apresentar cuidador	Israel, 49, Enfermagem	Dados do cuidador impressos	OK
Apresentar paciente	João, 39, contato Maria	Dados do paciente impressos	OK
Consultar estoque	Dipirona, estoque 10	Estoque atual: 10	OK
Estoque baixo	Estoque 10	False	OK
Registrar administração	Dipirona, Israel	Status Realizada, com data e hora	OK